



KOUJULURI VEERA BHASKARA DURGA PRASAD

MAIL ID: [kpurushotham9000@gmail.com](mailto:kpurushotham9000@gmail.com)

AWS DEVSECOPS-147

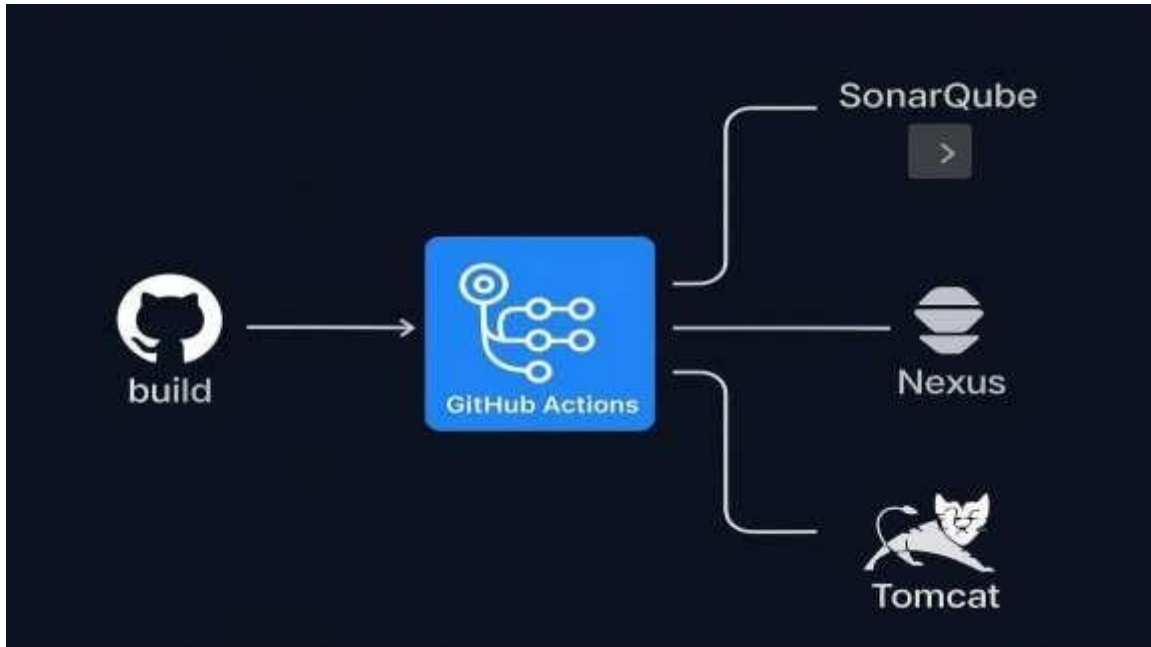
---

## End-to-End CI/CD Pipeline for Java Application Using GitHub Actions, SonarQube, Nexus & Apache Tomcat



## 1. Project Overview

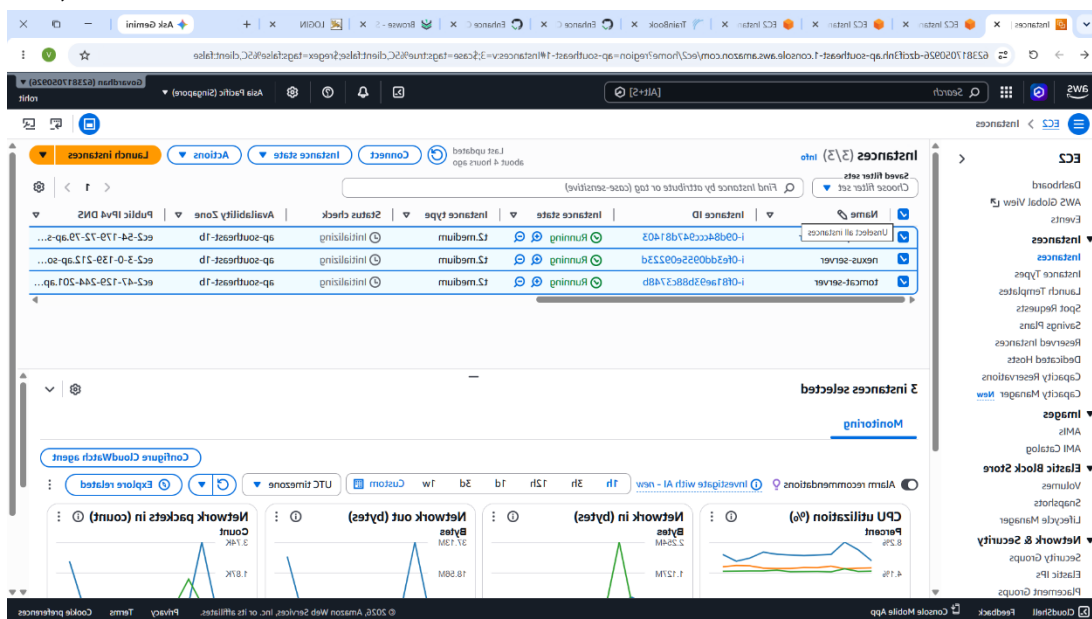
This project demonstrates a production-grade CI/CD pipeline built using GitHub Actions, integrated with SonarQube for code quality, Nexus Repository for artifact management, and Apache Tomcat for deployment.



The goal is to automate the entire build–test–analyze–deploy cycle for a Java-based web application.

## 2. High-Level Architecture

The architecture consists of three dedicated AWS EC2 instances running SonarQube, Nexus, and Tomcat.



Each component is isolated to ensure better scalability and maintainability.

### 3. AWS Infrastructure Setup

Three Ubuntu-based EC2 servers were launched:

- SonarQube Server — t2.medium
- Nexus Repository Server — t2.medium
- Tomcat Deployment Server — t3.micro

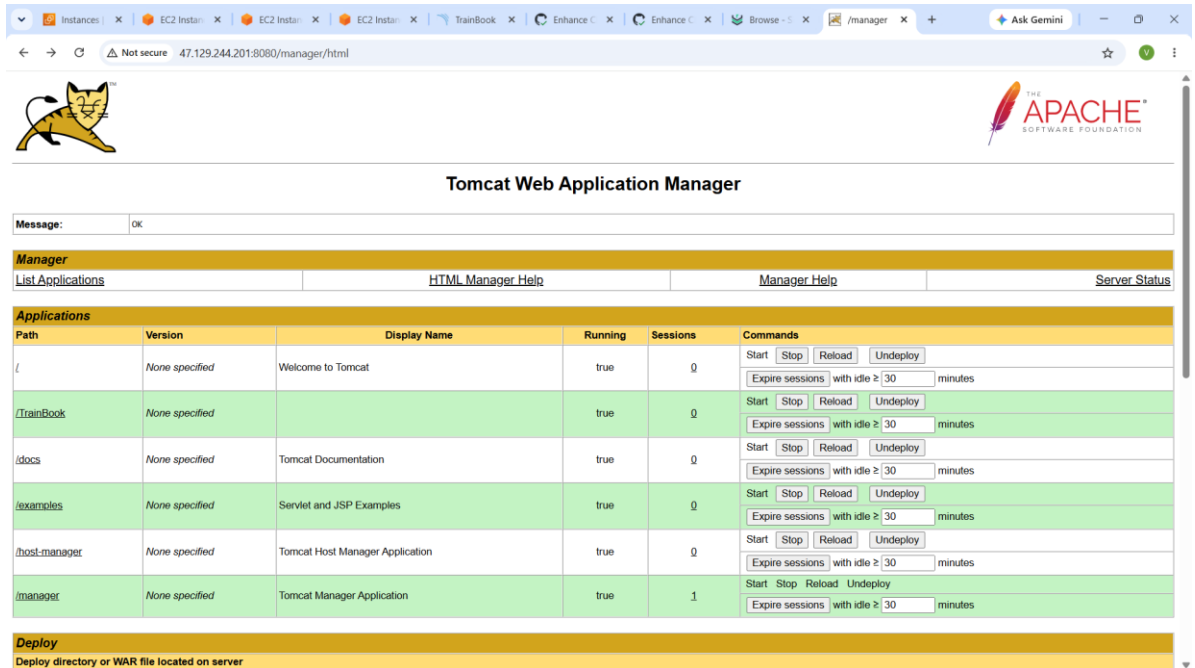
Security group rules include:

- Port 22 – SSH
- Port 9000 – SonarQube
- Port 8081 – Nexus
- Port 8080 – Tomcat

### 4. Tomcat Server Installation

Steps performed on Tomcat EC2:

1. Installed Java 11
2. Downloaded and extracted Apache Tomcat
3. Configured manager-gui and manager-script roles
4. Enabled remote deployment
5. Started Tomcat service.



## 5. SonarQube Installation

SonarQube was installed on a t2.medium EC2 with correct Java dependencies.

Steps performed on SonarQube:

1. copy the publicIP
2. paste the publicIP and port number in the web browser (IP:9000)
3. login to the SonarQube using user name(admin) and password (admin)

A project key and authentication token were generated for GitHub Actions integration.

## 6. Nexus Repository Setup

Nexus Repository Manager was installed and configured with both `maven-snapshots` and `maven-releases` repositories.

Steps performed on Nexus:

1. copy the publicIP
2. paste the publicIP and port number in the web browser (IP:8081)
3. login to the SonarQube using user name(admin)

4. For password: Go inside the Nexus container and go to the path - /nexus-data /admin.password.

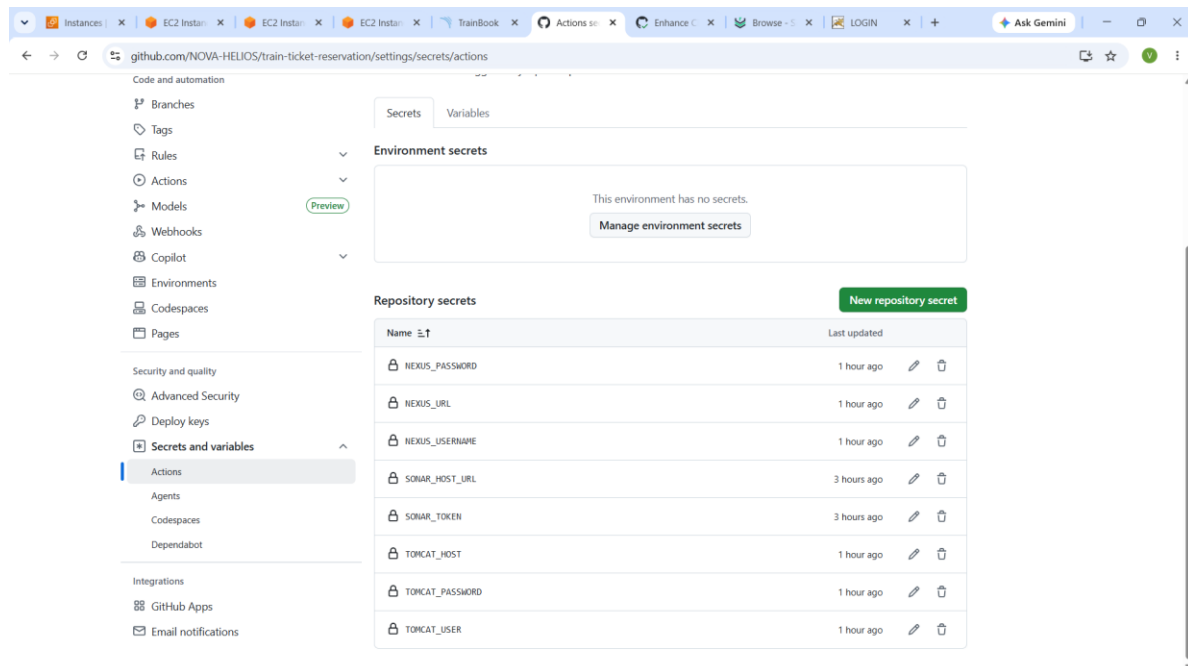
5. copy the password and paste it in the web browser Nexus console.

Admin credentials were updated and later stored as GitHub Secrets.

## 7. GitHub Secrets & Environment Variables

The following secrets were added to the GitHub repository:

- TOMCAT\_HOST
- TOMCAT\_USER
- TOMCAT\_PASSWORD
- SONAR\_HOST
- SONAR\_TOKEN
- NEXUS\_URL
- NEXUS\_USERNAME
- NEXUS\_PASSWORD



Instances x EC2 Instan x EC2 Instan x EC2 Instan x TrainBook x Actions 50 x Enhance C x Browse - x LOGIN x + Ask Gemini

github.com/NOVA-HELIOS/train-ticket-reservation/settings/secrets/actions

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security and quality

- Advanced Security
- Deploy keys
- Secrets and variables
- Actions
- Agents
- Codespaces
- Dependabot

Integrations

- GitHub Apps
- Email notifications

Secrets Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

New repository secret

| Name            | Last updated |
|-----------------|--------------|
| NEXUS_PASSWORD  | 1 hour ago   |
| NEXUS_URL       | 1 hour ago   |
| NEXUS_USERNAME  | 1 hour ago   |
| SONAR_HOST_URL  | 3 hours ago  |
| SONAR_TOKEN     | 3 hours ago  |
| TOMCAT_HOST     | 1 hour ago   |
| TOMCAT_PASSWORD | 1 hour ago   |
| TOMCAT_USER     | 1 hour ago   |

Instances x EC2 Instan x EC2 Instan x EC2 Instan x TrainBook x train-tick x Enhance C x Browse - x LOGIN x + Ask Gemini

github.com/NOVA-HELIOS/train-ticket-reservation/blob/master/github/workflows/main.yml

NOVA-HELIOS / train-ticket-reservation

Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Files

master

Go to file

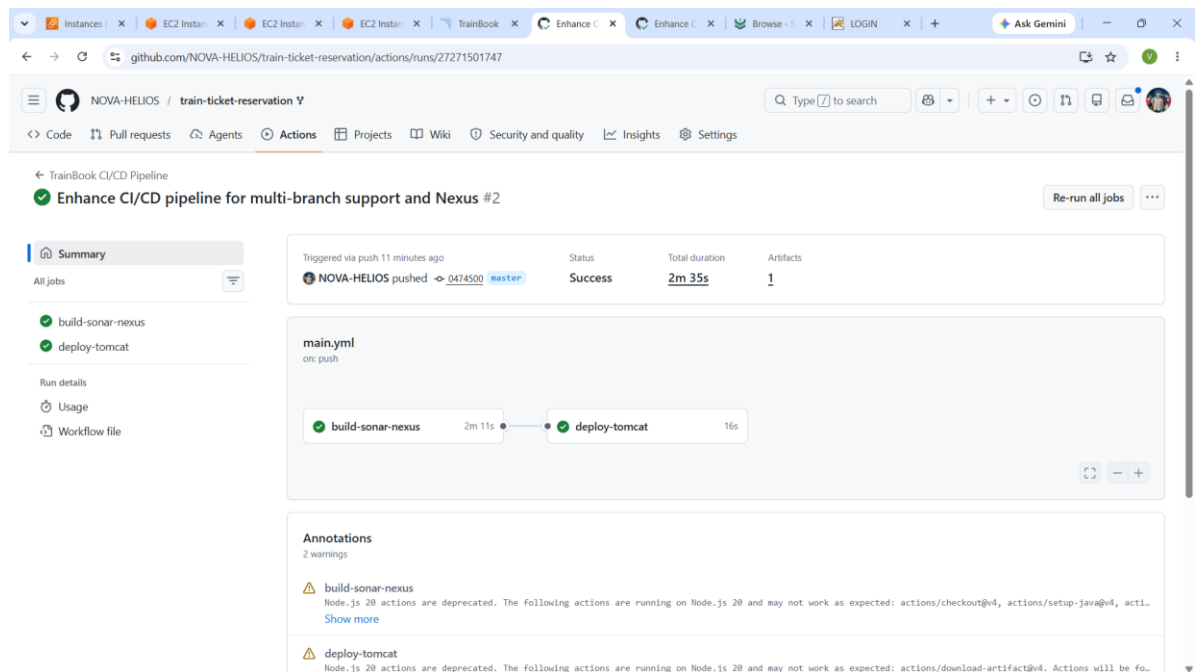
- .cache
- .github/workflows
- main.yml
- .ssh
- Screenshots
- WebContent
- src
  - .bash\_history
  - .bash\_logout
  - .bashrc
  - .profile
  - .sudo\_as\_admin\_successful
  - Dummy-Database.md
  - README.md
  - file1
  - pom.xml

train-ticket-reservation / github / workflows / main.yml

NOVA-HELIOS Enhance CI/CD pipeline for multi-branch support and Nexus 0474500 · 11 minutes ago History

Code Blame 88 lines (69 loc) · 2.4 KB

```
1 name: TrainBook CI/CD Pipeline
2
3 on:
4   push:
5     branches: ["master", "main"]
6
7 jobs:
8
9   # ----- BUILD + SONAR + NEXUS -----
10  build-sonar-nexus:
11
12    runs-on: ubuntu-latest
13
14    steps:
15
16      - name: Checkout Code
17        uses: actions/checkout@v4
18
19      # Java 17 FIX (required for SonarQube)
20      - name: Setup Java 17
21        uses: actions/setup-java@v4
22        with:
23          distribution: temurin
24          java-version: 17
25
```



## 8. CI/CD Pipeline Using GitHub Actions

The pipeline performs:

- Code checkout
- Java setup
- Maven build
- Unit tests
- SonarQube code analysis
- WAR packaging
- Artifact deployment to Nexus • Automated deployment to Tomcat name: ci/cd pipeline to deploy to tomcat

on:

push:

branch

es:

- master

jobs:        build-

deploy:

runs-on: ubuntu-latest

steps:

- name: checkout code

uses: actions/checkout@v3

- name: set-up jdk uses: actions/setup-java@v3 with:

distribution:    'temurin'    java-

version: '17'

# Create secure Maven settings.xml with Nexus creds

- name: Configure Maven settings for Nexus env:

NEXUS\_USERNAME: \${ secrets.NEXUS\_USER }

NEXUS\_PASSWORD: \${ secrets.NEXUS\_PASSWORD }

run: |

mkdir    -p    ~/.m2    cat    >

~/.m2/settings.xml    <<EOF

<settings



```
xmlns="http://maven.apache.org/SETTINGS/1.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
https://maven.apache.org/xsd/settings-1.0.0.xsd">
    <servers>
        <server>
            <id>nexus</id>
            <username>${NEXUS_USERNAME}</username>
            <password>${NEXUS_PASSWORD}</password>
        </server>
    </servers>
</settings>
EOF
```

- name: validate with maven run:

```
mvn validate
```

- name: compile with maven run:

```
mvn compile
```

- name: test with maven run:

```
mvn test
```

- name: package with maven run:

mvn package

- name: verify WAR file run: ls -lh

target/

- name: Build, Test and

SonarQube Scan run: | mvn

clean verify sonar:sonar \

-Dsonar.projectKey=train-ticket-reservation \

-Dsonar.host.url="\${{ secrets.SONAR\_HOST }}" \

-Dsonar.login="\${{ secrets.SONAR\_TOKEN }}"

# Deploy to Nexus using distributionManagement in POM

- name: Publish WAR to Nexus

run: mvn deploy

# Deploy WAR to Tomcat

- name: Deploy to tomcat run: |

curl -v -u "\${{

secrets.TOMCAT\_USER }}:\${{

secrets.TOMCAT\_PASSWORD

}}" \

-T target/TrainBook-1.0.0-SNAPSHOT.war \

```
"http://${ secrets.TOMCAT_HOST
}}:8080/manager/text/deploy?path=/TrainBook&update=true"
```

## 9. UPDATE POM.xml

Maven POM was updated for:

- Distribution Management
- Compiler and WAR plugins
- Sonar integration
- Nexus deployment

```
<project                                xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>


    <groupId>TrainBook</groupId>

    <artifactId>TrainBook</artifactId>

    <version>1.0.0-SNAPSHOT</version>

    <packaging>war</packaging>


    <name>TrainBook</name>


    <properties>

        <maven.compiler.source>17</maven.compiler.source>

        <maven.compiler.target>17</maven.compiler.target>
```

```
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>
```

```
<!-- This tells mvn deploy where to push -->
```

```
<distributionManagement>
```

```
  <snapshotRepository>
```

```
    <id>nexus</id>
```

```
    <url>http://34.227.67.34:8081/repository/maven-snapshots/</url>
```

```
  </snapshotRepository>
```

```
  <repository>
```

```
    <id>nexus</id>
```

```
    <url>http://34.227.67.34:8081/repository/maven-releases/</url>
```

```
  </repository>
```

```
</distributionManagement>
```

```
<build>
```

```
  <!-- You are using old style src/WebContent structure; keeping it so it doesn't break  
  -->
```

```
  <sourceDirectory>src</sourceDirectory>
```

```
<resources>
```

```
  <resource>
```

```
    <directory>src</directory>
```

```
    <excludes>
```

```
      <exclude>**/*.java</exclude>
```

```
</excludes>
</resource>
</resources>
<plugins>
<!-- Java compile -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>
    <source>${maven.compiler.source}</source>
    <target>${maven.compiler.target}</target>
  </configuration>
</plugin>

<!-- Build WAR from WebContent -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-war-plugin</artifactId>
  <version>3.4.0</version>
  <configuration>
    <warSourceDirectory>WebContent</warSourceDirectory>
  </configuration>
</plugin>
```

<!-- Optional: webapp-runner, you can keep or remove -->

<plugin>

<groupId>org.apache.maven.plugins</groupId>  
<artifactId>maven-  
dependency-plugin</artifactId>

<artifactId>maven-

<version>3.6.1</version>

<executions>

<execution>

<phase>package</phase>

<goals>

<goal>copy</goal>

</goals>

<configuration>

<artifactItems>

<artifactItem>

<groupId>com.github.jsimone</groupId>

<artifactId>webapp-runner</artifactId>

<version>8.0.30.2</version>

<destFileName>webapp-runner.jar</destFileName>

</artifactItem>

</artifactItems>

</configuration>

</execution>

</executions>

</plugin>

```
<!-- Snyk plugin (still there; run manually or add to pipeline) -->
<plugin>
  <groupId>io.snyk</groupId> <artifactId>snyk-maven-plugin</artifactId>
  <version>2.0.0</version>
  <inherited>>false</inherited>
  <configuration>
    <org>Venn1991</org>
  </configuration>
</plugin>
</plugins>
</build>
```

```
<dependencies>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.3.7</version>
  </dependency>
```

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.28</version>
</dependency>
```

```
<dependency>

  <groupId>javax.servlet</groupId> <artifactId>javax.servlet-api</artifactId>

  <version>3.1.0</version>

  <scope>provided</scope>

</dependency>

<!-- Add junit/junit-jupiter if you want proper tests -->

</dependencies>

</project>
```

## 10. Deployment Workflow

Each new commit triggers:

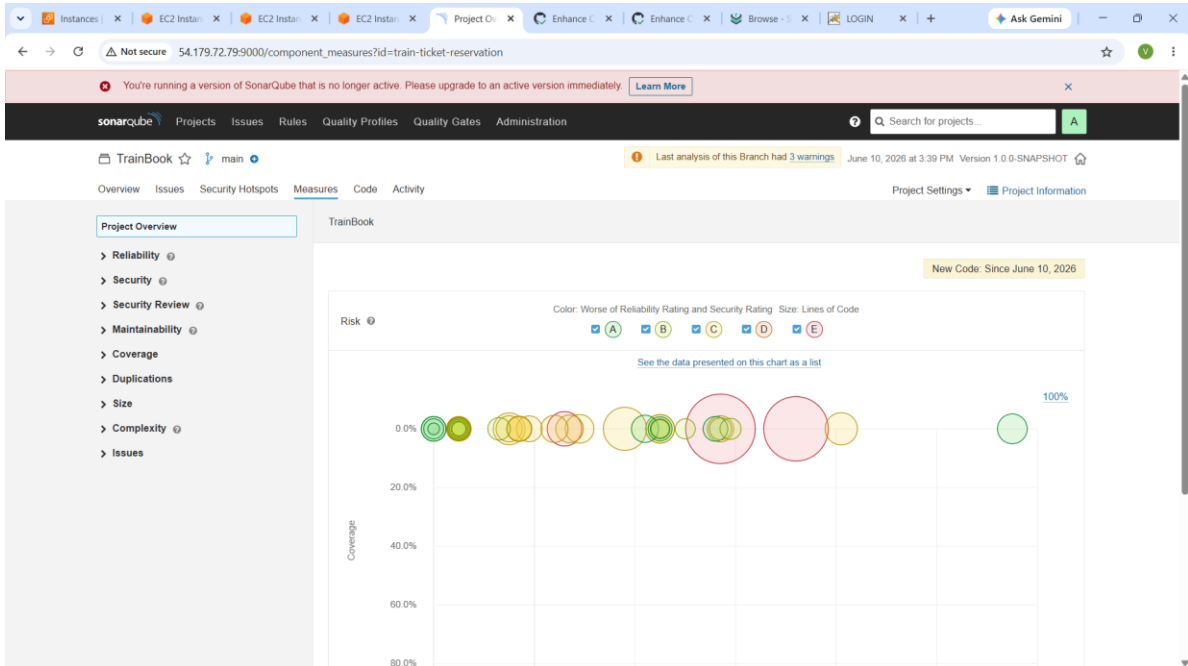
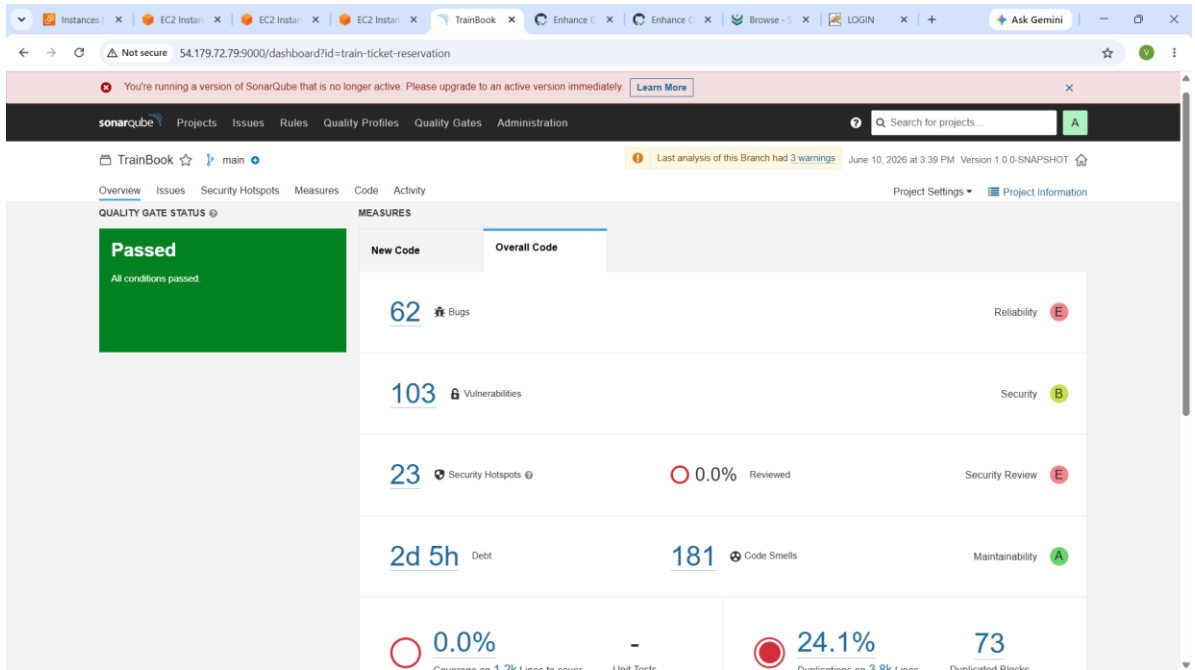
1. CI pipeline execution
2. SonarQube analysis
3. Nexus artifact upload
4. Tomcat remote deployment

The application becomes live immediately after deployment.

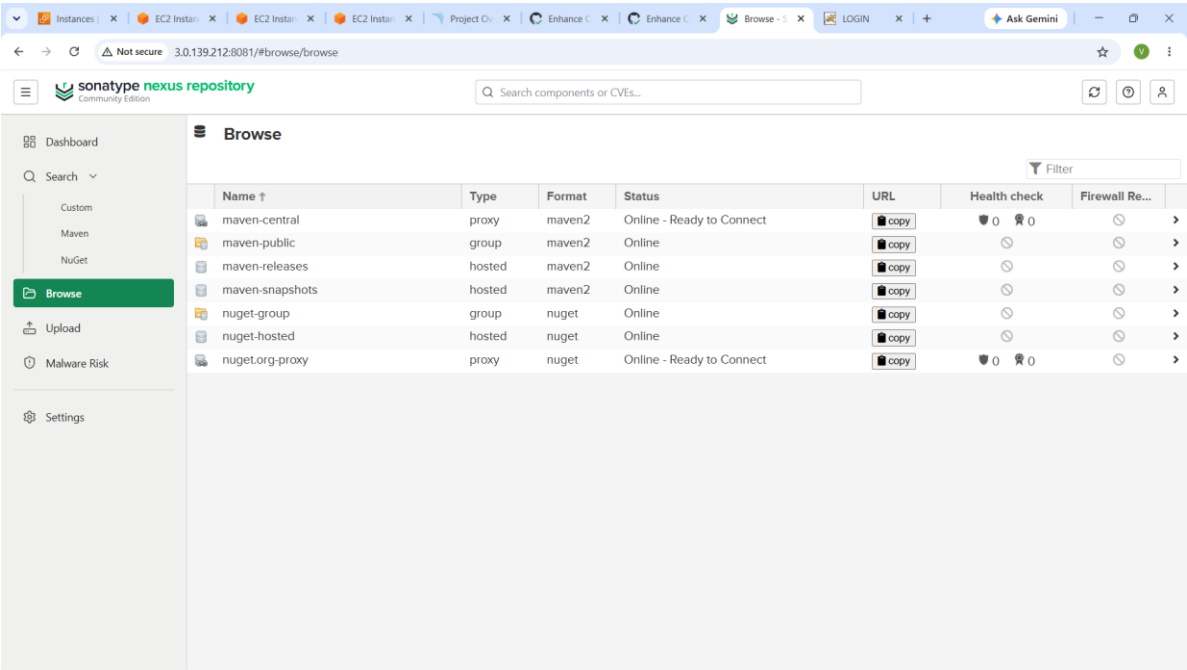
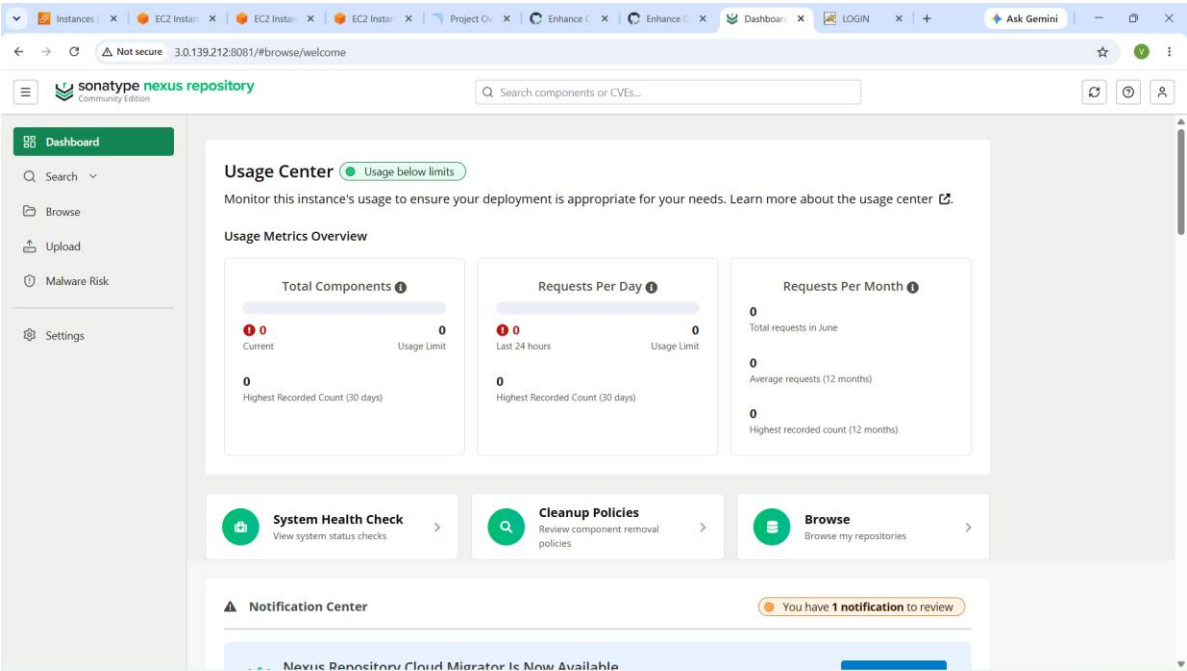
## 11. Final Output

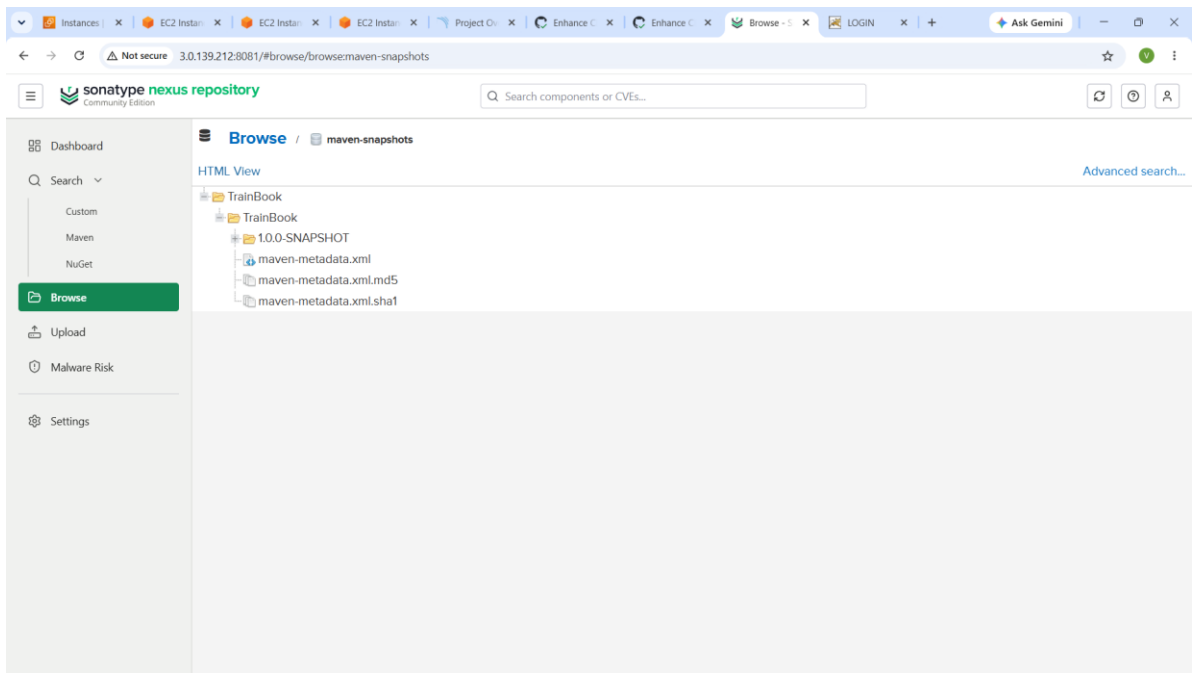
The SonarQube Analysis was successful:



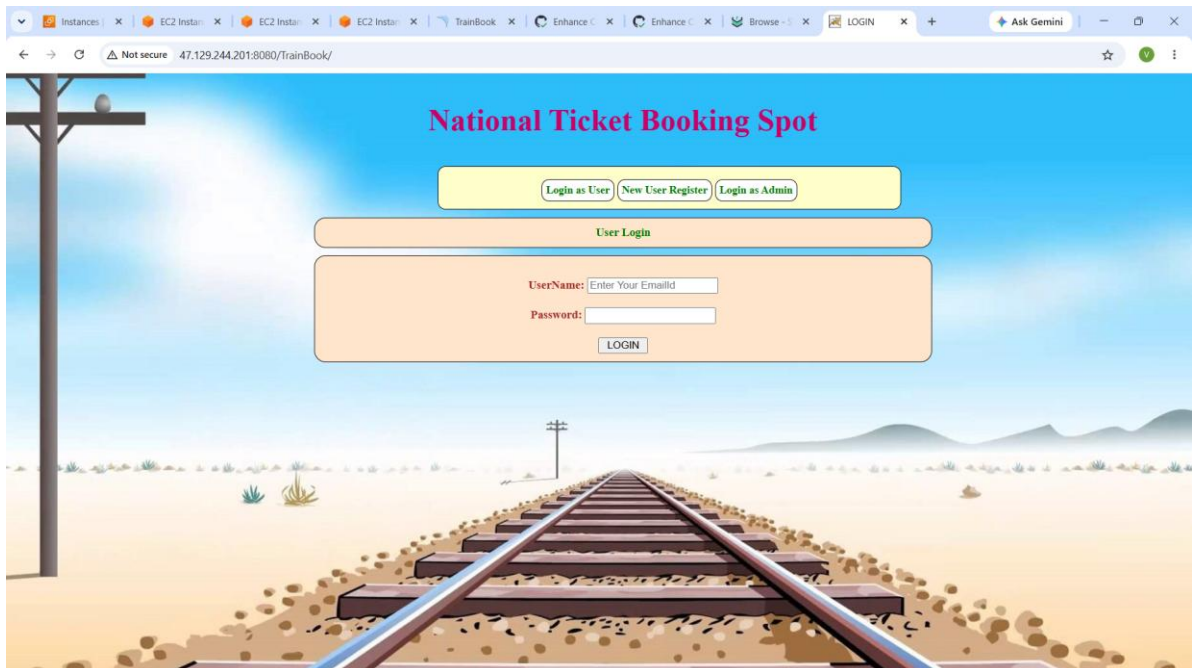


The Artifact successfully uploaded to Nexus:





The web applicationsuccessfullydeployed on Tomcat:



## 12. Conclusion

This project showcases a complete, production-ready CI/CD workflow integrating:

- Continuous Integration – automated builds, tests & security checks
- Continuous Delivery – artifact versioning & repository management
- Continuous Deployment – auto-deploying updates to Tomcat

It represents an industry-standard DevOps implementation suitable for scalable real-world environments.